



Faculty Of Information Science & Technology

TDM6323– Data Mining and Machine Learning

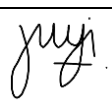
Trimester 3 – 2024 / 2025

Project Report

Title: Rental Pricing Dataset in Kuala Lumpur

Lab Session 1A

Group Number 2

	Student ID	Student Name	Programme	Contribution Percentage	Signature
1	1191202283	Uithiswary a/p Velan	BIA	25%	
2	1211203835	Liew Zhi Tong	BIA	25%	
3	1211204549	Priveeta Gobi Krishnan	BIA	25%	
4	1211201569	Yeo Hui Yi	BIA	25%	
		*Programme: AI / BIA / DCN / ITM / ST/BI			

Project Declaration Form

We hereby declare that all group members' names are correctly included in the section above and all members have contributed to this project. We hold a copy of this assignment, which we can produce if the original is lost or damaged. We certify that no part of this assignment has been copied from any other student's work or any other source except where due acknowledgement is made in the report.

We also acknowledge that the use of AI tools (if any) is aimed at brainstorming and enhancing decision-making processes. We affirm our commitment to ethical guidelines, transparency, and responsible utilization of AI technology in acquiring solutions.

Project Marking Scheme

1	Abstract		5
2	Introduction		5
3	Related Work		10
4	Dataset		10
5	Methodology/Framework		10
6	Experiments		5
7	Analysis and Discussion		10
8	Innovation and Originality		5
9	Conclusion		5
10	Reference & Format		5
11	Python Implementation		5
12	Presentation & QA		5
	Total Marks		80
	Final Marks		40

Group Member's Contribution to the Task

(Each group member, including the group leader, must individually fill out and submit this form. Elaborate on the tasks and contributions in this project. This form must be attached to the project submission.)

I, **Uithiswary a/p Velan** hereby declare that I have assessed the final submission, and I take full responsibility should there be any inaccuracies, incompleteness, omissions, delays, or non-submission. For the purpose of completing this project, we have worked as a team, and I have mainly contributed to the following tasks:

- I distributed the main task for each member for this project. I had taken the responsibility of writing up the methodology, experiments and analysis and discussion for the Gradient Boosting Algorithm. I compile the code and report for submission.

I, **LIEW ZHI TONG** hereby declare that I have assessed the final submission and I take full responsibility should there be any inaccuracies, incompleteness, omissions, delays, or non-submission. For the purpose of completing this project, we have worked as a team and I have mainly contributed in the following tasks:

- I had taken the responsibility of writing up the introduction for this report, furthermore, analyzed and assessed the machine learning model – Random Forest and completed the methodology, experiments and analysis for this model.

I, **PRIVEETA GOBI KRISHNAN** hereby declare that I have assessed the final submission, and I take full responsibility should there be any inaccuracies, incompleteness, omissions, delays, or non-submission. For the purpose of completing this project, we have worked as a team, and I have mainly contributed to the following tasks:

- I had taken the responsibility of writing up the abstract, and part of the related works and conclusion for this report, furthermore, analyzed and assessed the machine learning model – Neural Networks and completed the methodology, experiments and analysis for this model.

I, **YEO HUI YI** hereby declare that I have assessed the final submission, and I take full responsibility should there be any inaccuracies, incompleteness, omissions, delays, or non-submission. For the purpose of completing this project, we have worked as a team, and I have mainly contributed to the following tasks:

- I had taken the responsibility of writing up the dataset description for this report, furthermore, analyzed and assessed the machine learning model – K-Means Clustering and completed the methodology, experiments and analysis for this model.

Abstract

The project investigates rental market patterns in Kuala Lumpur and Selangor, Malaysia to understand landlord and tenant difficulties in establishing suitable rents. A detailed breakdown of Malaysian rental prices presents through the "Rental Pricing Dataset, Malaysia" reveals how property type alongside location and available amenities direct market rates. The research uses Python for exploratory data analysis (EDA) together with Random Forest, Neural Networks K-Means Clustering and Gradient Boosting machine learning models to analyze the information and detect patterns throughout the dataset. The dataset received extensive cleaning and preprocessing before different machine learning methods were used for both rental price prediction and classification tasks. The study reveals that rental prices significantly relate to apartment dimensions together with construction year and unit space quantity. The research contributes improved knowledge about rental price formation while delivering actionable data to tenants and landlords and policymakers to drive wiser choices and increase urban housing affordability.

1.0 Introduction

The rental markets in Kuala Lumpur and Selangor are important for Malaysia's urban economies. They have high demand for housing and are growing fast. Even with online platforms like mudah.my to search for properties, landlords and tenants still face problems in determining fair rental prices. The lack of transparency in the rental market creates confusion about suitable prices. This leads to wrong decisions between renters and landlords. It may also cause price distortions in the market.

This project aims to solve these problems. It uses the "Rental Pricing Dataset, Malaysia," which contains detailed information about rental properties in Kuala Lumpur and Selangor. The study looks at factors that affect rental prices, such as property type, location, and amenities. It also examines how these factors change across different areas. By employing Python-based exploratory data analysis (EDA) techniques and implementing advanced machine learning models— Random Forest (Classification), Neural Network, K-Means Clustering, and Gradient Boosting—the project provides a comprehensive analysis of rental market dynamics.

The main objectives of this project include preparing the dataset by fixing missing values, removing outliers, and making sure it is consistent. In addition, descriptive statistics and visualizations are used to find trends and relationships. Machine learning models are applied to find the most important factors affecting rental prices. Recommendations are made based on the results to help stakeholders.

Through its innovative use of machine learning, this project helps close the information gap in the rental market. It provides a data-driven way to understand price patterns. It also offers solutions to improve rental decisions for everyone involved. The findings aim to support tenants in finding affordable housing, assist landlords in setting competitive rental prices, and help policymakers address urban housing affordability.

In conclusion, this project not only broadens our understanding of factors shaping rental prices but also establishes a framework for fair and transparent decision-making in the rental markets of Kuala Lumpur and Selangor.

2.0 Related works

No	Authors	Year	Title	Objectives	Results	Findings
1.	Tayo P Ogundunmade , Muiyiwa Abidoye, Oladapo M Olunfunbi	2023	Modelling Residential Housing Rent Price Using Machine Learning Models	This study examines the factors affecting residential housing rent in Ibadan City to thoroughly	The five variables with the highest correlation coefficients were identified as	- Factors such as the type of house, presence of a toilet, location, number of bedrooms,

				investigate the cost-effectiveness of rents. We looked into a number of variables, including neighborhood characteristics, property size, amenities, and location, aiming to pinpoint the essential elements significantly impacting rental costs.	duplex (0.54), toilet (0.34), detached (0.31), flat (-0.3), and bedroom (0.29), indicating their substantial influence on housing rental rates. Furthermore, the study highlights specific neighborhoods as either relatively expensive or budget-friendly in terms of median rent prices. Notably, Agodi and Kolapo Ishola emerged as the most expensive areas, surpassing the overall median rent by significant margins (454.55% and 450%, respectively). Conversely, Sanyo, Moniya, and Academy recorded the lowest median rent prices, being 50% lower than the	and whether the house is a flat have the most significant impact on house rent prices in Ibadan. - The most expensive neighborhoods in Ibadan are Kolapo Ishola, Jericho, Apata Alakia, Agodi, Bodija, Ologuneru, Apata, and Agodi, with median rent prices significantly higher than the overall median. - The XGBoost model demonstrated strong predictive performance in estimating house rent prices, outperforming the OLS regression model.
--	--	--	--	---	---	---

					overall median.	
2.	Asif Ahmed Nelay, H.M. Sadman Haque, Md. Mahmud UI Islam	2019	Ensemble Learning Based Rental Apartment Price Prediction Model by Categorical Features Factoring	- Develop a model to predict apartment rental prices based on various features - Identify the key factors that influence apartment rental prices in Dhaka, Bangladesh - Provide a tool that can be used by real estate agents and renters to determine optimal apartment prices and match renters with suitable apartments	The key result of the study is that the ensemble gradient boosting model achieved the highest prediction accuracy of 88.75% for predicting apartment rental prices in Dhaka, Bangladesh. The paper also found that categorical features and their interactions with other variables, particularly size and total rooms, are important for accurately predicting rental prices.	- The ensemble learning approach, specifically Gradient Boosting, achieved the highest accuracy of 88.75% in predicting apartment rental prices. - Categorical features in the dataset, when converted to dummy variables, had a significant impact on the model's performance. - The size and total number of rooms in an apartment were the most important features in determining the rental price.
3.	Shamsideen Olalekan, Oshodi, Wellington Didibhuku Thwala, Tawakalitu Bisola Odubiyi, Rotimi Boluwatife Abidoye, Clinton Ohis Aigbavboa	2019	Using neural network model to estimate the rental price of residential properties	-To evaluate the efficacy of using neural network model for estimation of the rental price of residential properties - To examine the impact of the attributes of a residential property on its value.	In the light of these results, the developed neural network model could be used to estimate rental price for taxation. Also, the significant variables identified need to be included in the designs of new	The results show that the predictive accuracy of the developed neural network model is 78.95 per cent. Based on the sensitivity analysis of the model, it was revealed that balcony and floor area have the most significant impact on the

					residential homes and this would ensure optimal returns to the investors.	rental price of residential properties. However, parking type and swimming pool had the least impact on rental price. Also, the availability of garden and proximity of police station had a low impact on rental price when compared to balcony.
4.	Chuang Hu, Rui Huang, Haijian Li	2023	Prediction and Analysis of Rental Price using Random Forest Machine Learning Technique Take Shanghai and Wuhan for example	- To study the price trends and influencing factors in the rental markets of Shanghai and Wuhan - To analyze the factors affecting rental prices and provide price forecasts to help tenants and landlords make better decisions - To provide a reference for future researchers studying the housing rental market	The paper applies the random forest model to forecast the rental house prices in Shanghai and Wuhan, China, and analyzes the factors that affect the rental house prices in these two cities.	- The random forest model had a poor prediction effect on rental prices in Shanghai, despite the selected variables having a good explanatory effect. - The random forest model had a significantly better prediction effect on rental prices in Wuhan, with a high degree of fit between the predicted and actual prices. - The area within the city and housing features like floor area and elevator

						equipment had a significant impact on rental prices in Wuhan.
5.	Kokasih, Marco Febriadi Paramita, Adi Suryaputra	2020	Property Rental Price Prediction Using the Extreme Gradient Boosting Algorithm	The objective of the study is to develop a prediction model for property rental prices using Extreme Gradient Boosting (XGBoost) algorithm. The model aims to assist property owners and tourists to determine fair rental prices based on listing features, neighborhood characteristics and host information.	The study used a dataset from the Inside Airbnb project which focus on Singapore Airbnb listings. The model was trained using an 80:20 train-test split ratio and evaluated using Root Mean Square Error (RMSE) as the performance metric. The result from the study shows an average RMSE of 10.86, translating to an accuracy of approximately 13.30%. The highest RMSE recorded was 38.49% in one test case, while the lowest was 0.99% in another.	In this study, the key findings aspect are effectiveness of XGBoost, feature importance and implementation potential.

The reviewed literature demonstrates multiple prediction techniques and various outcomes within different geographic areas. XGBoost surpassed traditional regression models according to

Ogundunmade et al. (2023) while considering property type and location variables and amenities in Ibadan Nigeria's market. A study by Neloy et al. (2019) in Dhaka Bangladesh used ensemble learning methods to achieve 88.75% accuracy rates with apartment size and room count among key categorical features that shaped the results. Neural networks used by Olalekan et al. (2019) demonstrated effective rental price prediction through floor area and balcony identification as key determinants yet swimming pool and parking attributes proved negligible. The research by Hu et al. (2023) used Random Forest models to analyse Shanghai along with Wuhan where urban position and housing characteristics affected rental prices with Wuhan showing superior predictive accuracy. As reported by Kokasih & Suryaputra Paramita (2020) XGBoost successfully extracted complex patterns from Singapore Airbnb data however presented inconsistent test performance. Collectively, these studies underline the importance of advanced machine learning models, feature selection, and regional characteristics in improving rental price prediction accuracy.

3.0 Dataset Description

This dataset consists of property rental listings in Kuala Lumpur and Selangor. It is extremely relevant to the analysis of real estate trends. The dataset contains crucial information that enables an in-depth investigation of the variables affecting rental prices and property preferences. Key characteristics that are essential for assessing the dynamics of the rental market include property type, location, size, monthly rent, and furnishing status. These factors allow for a more thorough understanding of the supply and demand in the market by revealing information about the kinds of property that are available, their prices, and the provided facilities.

There is a variety of property factors in the dataset, such as the number of rooms, bathrooms and parking spots, enhances the analysis by enabling us to investigate the ways in which qualities impact rental desirability. Metrics like completion year, which allow us to evaluate how property age affects rental preferences and pricing.

This dataset is a great choice due to its comprehensive coverage of factors that are crucial for examining the rental market. Its detailed structure guarantees solid analysis and closely matches our project's goals. This dataset offers the basis for significant insights into market trends and property preferences in Kuala Lumpur and Selangor by concentrating on important elements that influence renting decisions.

Table 3.1 Data Dictionary

Attributes	Description
ads_id	Unique identifier of each listing
prop_name	Property name
completion_year	Completion year of property
monthly_rent	Monthly rental price
location	Location of property
property_type	Type of property
rooms	Number of rooms
parking	Number of parking spaces
bathroom	Number of bathrooms
size	Size of property
furnished	Furnishing status
facilities	Available facilities
additional_facilities	Additional facilities
region	Region of property

3.1 Data Cleaning

We implemented an organized data cleaning techniques to guarantee the accuracy, consistency and reliability of the dataset. Missing values in numeric columns like completion_year, parking, and bathroom were imputed using the median to preserve data consistency and reduce the impact of outliers. While missing values for categorical columns such as prop_name, furnished, and facilities were substituted with the mode to maintain logical consistency throughout the dataset.

Furthermore, formatting errors were fixed to guarantee that all data was in a usable format. For instance, non-numeric characters were removed from the monthly_rent column so that only numeric values remained and were converted to integers. Columns including size, rooms, bathroom, and parking were processed to extract numeric values from text-based entries, ensuring data type consistency for analysis. Additionally, temporal analyses were made possible by converting the completion_year field to integers.

We fixed the problems of missing data, inconsistent formatting, and type mismatches with these cleaning steps. As a result, the dataset was clear and trustworthy, offering a strong basis for precise analysis.

3.2 Descriptive Statistics

	count	mean	std	min	25%	50%	75%	max
ads_id	6043.0	9.975880e+07	3.321355e+06	16717632.0	99851539.5	100181825.0	100597829.5	100854262.0
completion_year	6043.0	2.014969e+03	6.790393e+00	1980.0	2013.0	2017.0	2020.0	2025.0
monthly_rent	6043.0	2.775019e+03	3.501497e+04	290.0	1300.0	1600.0	2000.0	2400000.0
rooms	6043.0	2.667549e+00	8.427413e-01	1.0	2.0	3.0	3.0	9.0
parking	6043.0	1.465994e+00	5.636809e-01	1.0	1.0	1.0	2.0	10.0
bathroom	6043.0	1.925534e+00	5.678828e-01	1.0	2.0	2.0	2.0	8.0
size	6043.0	9.601547e+02	6.951678e+02	1.0	800.0	921.0	1096.0	48010.0

Table 3.2.1 shows descriptive statistics on rental pricing dataset

Table 3.2.1 shows descriptive statistics for rental pricing dataset which include 6,043 entries with 0 missing values. The completion_year column reveals that the properties were completed between 1980 to 2025. The 25th percentile for completion_year is 2013 and the 75th percentile for completion_year is 2020. The median for completion_year is 2017. Other than that, the monthly_rent column has the minimum rent of RM290 and the maximum rent of RM2,400. The 25th percentile for monthly_rent is RM1,300 and the 75th percentile is RM2,000. The median for monthly_rent is RM1,600.

The rooms attribute shows that the number of rooms ranges from 1 to 9 with median of 3 rooms. The 25th percentile for rooms shows 2 and the 75th percentile. For parking attribute, the minimum number of parking is 1 and maximum number of parking is 10 with median for parking attribute is 1. The 25th percentile for parking is 1 and 75th percentile is 2. The bathroom column shows a range of 1 to 8 bathrooms with a median of 2 bathrooms. The 25th percentile and 75th percentile is 2 bathrooms. Property size ranges from 1 to 48,010sqft. Most of the properties falls between 800 to 1,096sqft in size with a median of 921sqft.

The dataset displays a wide range of property types, reflecting various properties with respect to completion year, size, rental prices, and features. The majority of properties have moderate sizes, ranging from 2 bedrooms and 1-2 bathrooms to 1-2 parking spaces. The variations in monthly rents and sizes suggest a mixture of both some smaller, more affordable properties as well as larger, high-end listings.

4.0 Methodology

4.1 Random Forest Algorithm

The first methodology for this project uses the Random Forest algorithm for classification. It is a strong ensemble learning method to analyze apartment rental data and predict the price category of monthly rent (Hu et al., 2023). The workflow starts with data preprocessing to ensure data quality and consistency. Key steps include turning non-numeric columns like size, rooms, and parking into integers by extracting numbers. Missing values are handled by filling them with medians. Categorical variables are converted into numbers using one-hot encoding. The target variable, "monthly_rent," was divided into five bins to create distinct price ranges. This step turned the continuous target variable into a classification problem with predefined categories.

The dataset was split into training and testing sets in an 80-20 ratio. This ensured this methodology enabled a fair assessment of the model. The Random Forest Classifier delivered optimal results for handling data sets with many dimensions. The approach provides ways to estimate which aspects contribute to the most relevant input variables. The training process used decision trees numbering 100 to work on the training data. Performance enhancement and overfitting minimization were achieved through ensemble learning methods.

The model performance was evaluated using three metrics: confusion matrix alongside classification report and accuracy score. Feature importance analysis was also done to find the most important factors for predictions. This helped make the results more understandable and gave useful insights.

4.2 Neural Network

We have also utilized Neural Networks to forecast apartment prices through the identification of sophisticated feature connections. As an inspiration from human cerebral functions neural networks arrange node layers connected by trainable weights to decrease forecast mistakes. When setting up the methodology the research team selected properties based on their dimension size and positioning and the building type as well as amenities. A normalization procedure using "StandardScaler" standardized numerical features while the categorical attributes received encoding to prepare them for model incorporation.

We developed the neural network with TensorFlow and Keras using densely connected building blocks. Neurons composed the first network layer at five while representing the chosen features. The model implemented ReLU activation functions across two hidden layers which contained 64 and 32 neurons to introduce non-linearity in the network. A single output neuron with linear activation determined the model's prediction of continuous rental prices. Adam optimizer in combination with MSE as the main loss function and MAE served as a supplement during validation along the training process.

Warning against overfitting data we used 20% of the training set for validation and employed loss and performance plots for monitoring the training execution. Monitoring for training and validation loss pattern shifts occurred as part of routine assessment procedures. Actual and predicted rental prices were plotted on scatter graphs to check prediction accuracy levels.

4.3 K-Means Clustering

Additionally, the third model we chose to use in our project is K-Means Clustering. The features included monthly rent, property size, number of rooms, bathrooms and parking spaces. First, monthly_rent, size, rooms, bathroom and parking column have been cleaned and converted to integer for analysis. Then, all features have been standardized using StandardScaler. Next, split the dataset into training and testing sets with an 80:20 ratio. Models were trained with K values ranging from 2 to 10.

The K-Means algorithm was applied to group data points based on feature similarity. The number of clusters (k) was chosen using the Elbow Method. This method finds where the curve of inertia starts to flatten. Silhouette Scores used to evaluate how well the K-Means clustering results are. After clustering, R-squared accuracy was computed to measure how many variances in the data were explained by the cluster centers. This helped check how well the clusters represented the data.

The Elbow Method and Silhouette Scores were visualized to select the best k . Cluster distributions were plotted to see how well the clusters were separated. Tuned and intermediate models were also explored to enhance performance.

4.4 Gradient Boosting

The last model we choose to use in our project is gradient boosting algorithm. The dataset we use is the apartment listings-related feature collection. Firstly, the data is cleaned for processing with missing values, outliers, and incoherent values. Categorical values, like one-hot encoded and label encoded, will be converted into numerical values. Feature scaling will be performed for feature normalizing, which will proportionately contribute to the performance of the model. The dataset will be partitioned into training and testing sets in an 80:20 proportion for post preprocessing. 80% of the data is a training set that will be used for training the gradient boosting model, and 20% of the test data will be used for testing and valuation. The partitioning model will validate with unseen samples in an attempt not to overfit and provide a correct estimation of its generalization performance.

The GBM model initiated with important hyperparameters, such as the number of boosting rounds ($n_estimators$), its learning rate, and its individual tree's deepness. The hyperparameters were tuned with initial experiments and expert experiences for the best performance achievement. The GBM algorithm sequentially integrates decision trees, and a new tree corrects the errors of its predecessor. Iterating continues to satisfactory model performance, or stop criterion reached. Performance of the model in training is monitored with performance measures such as mean squared error (MSE) or root mean squared error (RMSE) for a problem that is a regression, or accuracy, precision, recall, and $f1$ -score for a problem that is a classification. Cross-validation techniques such as k -fold cross-validation are implemented for uniform performance over many training data sets and for overfit checking.

After training, testing with the testing set was conducted in an objective to evaluate its generalization performance. Performance statistics in the testing set provided an estimation towards the performance of the model with new, unseen observations. The calculation of feature importance values was conducted to know the most contributing features towards model prediction, providing insightful information towards the pattern in the data present under it. Model performance was evaluated with baseline models, a decision tree, or a simple model, in an objective to evaluate that the GBM technique did a better job in comparison with them. In a case of a necessity, additional hyperparameter tuning with grid search or random search was performed.

5.0 Experiments

5.1 Random Forest Experimental Setup

The Random Forest classifier analysed residential rental prices throughout Kuala Lumpur, Malaysia through different apartment characteristics. To prepare the dataset, missing values were handled, categorical variables were encoded, and the target variable ($monthly_rent$) was binned into five rental price categories:

- Class 0: Rentals below RM1500,
- Class 1: Rentals between RM1500 and RM3000,
- Class 2: Rentals between RM3000 and RM4500,
- Class 3: Rentals between RM4500 and RM6000,
- Class 4: Rentals above RM6000.

The dataset was split into an 80% training set and a 20% test set, with stratified sampling to maintain class proportions. A grid search was conducted to optimize the model's hyperparameters. The following parameter ranges were tested

- Number of estimators ($n_estimators$): [50, 100, 200],
- Maximum depth of the trees (max_depth): [None, 10, 20, 30],

- Minimum samples required to split a node (min_samples_split): [2, 5, 10], and
- Minimum samples required in a leaf node (min_samples_leaf): [1, 2, 4].

A total of 108 hyperparameter combinations were evaluated. The records of accuracy from every combination turned into data used to recognize the best-performing model.

5.1.1 Hyperparameter Tuning Results

A grid search identified the top-performing model which delivered 86.69% accuracy for the test data. The optimal hyperparameters were:

- n_estimators: 200,
- max_depth: None,
- min_samples_split: 2,
- min_samples_leaf: 1.

This configuration was used to train the final Random Forest model. The visualization of hyperparameter tuning results is shown in Figure 5.1.1.1.

```
Best Hyperparameters:
n_estimators      200.000000
max_depth         NaN
min_samples_split  2.000000
min_samples_leaf  1.000000
accuracy          0.866933
Name: 72, dtype: float64
```

Figure 5.1.1.1 Impact of Hyperparameters on Model Accuracy

Figure 5.1.1.1 shows the impact of multiple hyperparameter combinations on the model's accuracy. Higher accuracies were achieved with a larger number of estimators (n_estimators) and deeper tree structures (max_depth: None), indicating that a more complex model was better suited for this dataset.

5.2 Neural Network experiment setup

```
Epoch 1/100 ----- 2s 5ms/step - loss: 387299328.0000 - mae: 2578.4341 - val_loss: 351888328.0000 - val_mae: 2379.7943
Epoch 2/100 ----- 1s 4ms/step - loss: 211390216.0000 - mae: 2178.3755 - val_loss: 351839048.0000 - val_mae: 2318.8389
Epoch 3/100 ----- 0s 3ms/step - loss: 317916416.0000 - mae: 2468.1648 - val_loss: 351831088.0000 - val_mae: 2168.9187
Epoch 4/100 ----- 1s 4ms/step - loss: 458937472.0000 - mae: 2486.3223 - val_loss: 358228048.0000 - val_mae: 1938.8417
Epoch 5/100 ----- 1s 3ms/step - loss: 226189824.0000 - mae: 1748.2549 - val_loss: 349244864.0000 - val_mae: 1697.5898
Epoch 6/100 ----- 1s 2ms/step - loss: 249358884.0000 - mae: 1681.2866 - val_loss: 348378928.0000 - val_mae: 1489.9734
Epoch 7/100 ----- 0s 2ms/step - loss: 388998888.0000 - mae: 1452.3199 - val_loss: 347883584.0000 - val_mae: 1324.3281
Epoch 8/100 ----- 0s 2ms/step - loss: 286485448.0000 - mae: 1294.4289 - val_loss: 347139984.0000 - val_mae: 1289.9464
Epoch 9/100 ----- 0s 2ms/step - loss: 316495392.0000 - mae: 1198.5187 - val_loss: 348761824.0000 - val_mae: 1154.5322
Epoch 10/100 ----- 1s 2ms/step - loss: 87381392.0000 - mae: 725.1838 - val_loss: 348526432.0000 - val_mae: 1141.1482
Epoch 11/100 ----- 0s 2ms/step - loss: 289781928.0000 - mae: 1848.5814 - val_loss: 348277856.0000 - val_mae: 1161.5555
Epoch 12/100 ----- 0s 2ms/step - loss: 56428892.0000 - mae: 1666.6388 - val_loss: 348163872.0000 - val_mae: 1182.8546
Epoch 13/100 ----- 0s 2ms/step - loss: 167528888.0000 - mae: 958.8815 - val_loss: 348878848.0000 - val_mae: 1211.6553
Epoch 14/100 ----- 0s 3ms/step - loss: 221781512.0000 - mae: 1864.4537 - val_loss: 348823328.0000 - val_mae: 1238.7828
Epoch 15/100 ----- 1s 2ms/step - loss: 195895248.0000 - mae: 1817.5838 - val_loss: 345997824.0000 - val_mae: 1237.9881
Epoch 16/100 ----- 1s 2ms/step - loss: 368848968.0000 - mae: 1859.5845 - val_loss: 345943744.0000 - val_mae: 1267.2324
Epoch 17/100 ----- 0s 2ms/step - loss: 341529472.0000 - mae: 1469.3777 - val_loss: 345914816.0000 - val_mae: 1286.8531
Epoch 18/100 ----- 0s 2ms/step - loss: 118483824.0000 - mae: 1845.4378 - val_loss: 345888488.0000 - val_mae: 1294.1356
Epoch 19/100 ----- 0s 2ms/step - loss: 118487488.0000 - mae: 937.4371 - val_loss: 345871648.0000 - val_mae: 1312.8562
Epoch 20/100 ----- 1s 2ms/step - loss: 438513824.0000 - mae: 1639.5176 - val_loss: 345854384.0000 - val_mae: 1319.6738
Epoch 21/100 ----- 0s 2ms/step - loss: 295856168.0000 - mae: 1375.8816 - val_loss: 345832384.0000 - val_mae: 1326.3817
Epoch 22/100 ----- 0s 2ms/step - loss: 167378824.0000 - mae: 1166.2968 - val_loss: 345813376.0000 - val_mae: 1327.7781
Epoch 23/100 ----- 0s 2ms/step - loss: 43128624.0000 - mae: 1549.6278 - val_loss: 345798728.0000 - val_mae: 1341.2328
Epoch 24/100 ----- 0s 2ms/step - loss: 327884864.0000 - mae: 1482.1897 - val_loss: 345783712.0000 - val_mae: 1344.8887
Epoch 25/100 -----
```

Figure 5.2.1 Epoch 1-24

```

Epoch 25/100      0s 2ms/step - loss: 180857824.0000 - mae: 1421.4139 - val_loss: 345772200.0000 - val_mae: 1348.4139
121/121 -----
Epoch 26/100      0s 2ms/step - loss: 180851344.0000 - mae: 1468.4247 - val_loss: 345763680.0000 - val_mae: 1334.1343
121/121 -----
Epoch 27/100      0s 2ms/step - loss: 175683968.0000 - mae: 1894.4272 - val_loss: 345758048.0000 - val_mae: 1351.8026
121/121 -----
Epoch 28/100      0s 2ms/step - loss: 49348016.0000 - mae: 1739.1287 - val_loss: 345741600.0000 - val_mae: 1352.2899
121/121 -----
Epoch 29/100      0s 2ms/step - loss: 223875600.0000 - mae: 1190.4751 - val_loss: 345727424.0000 - val_mae: 1352.3639
121/121 -----
Epoch 30/100      0s 2ms/step - loss: 155180304.0000 - mae: 993.9857 - val_loss: 345722824.0000 - val_mae: 1353.2688
121/121 -----
Epoch 31/100      0s 2ms/step - loss: 287922544.0000 - mae: 1215.2968 - val_loss: 345699136.0000 - val_mae: 1349.2994
121/121 -----
Epoch 32/100      0s 2ms/step - loss: 188372448.0000 - mae: 1185.8145 - val_loss: 345788048.0000 - val_mae: 1377.7954
121/121 -----
Epoch 33/100      0s 2ms/step - loss: 277248064.0000 - mae: 1453.8564 - val_loss: 345781824.0000 - val_mae: 1379.6632
121/121 -----
Epoch 34/100      0s 2ms/step - loss: 277086432.0000 - mae: 1284.8688 - val_loss: 345692960.0000 - val_mae: 1371.8634
121/121 -----
Epoch 35/100      0s 3ms/step - loss: 381274400.0000 - mae: 1532.6758 - val_loss: 345689176.0000 - val_mae: 1373.0884
121/121 -----
Epoch 36/100      1s 3ms/step - loss: 23482728.0000 - mae: 1234.7283 - val_loss: 345682616.0000 - val_mae: 1385.8593
121/121 -----
Epoch 37/100      1s 3ms/step - loss: 238646728.0000 - mae: 1263.9645 - val_loss: 345678932.0000 - val_mae: 1387.7758
121/121 -----
Epoch 38/100      1s 4ms/step - loss: 123889184.0000 - mae: 1628.9818 - val_loss: 34565248.0000 - val_mae: 1379.5939
121/121 -----
Epoch 39/100      0s 3ms/step - loss: 156337568.0000 - mae: 1139.4984 - val_loss: 345672028.0000 - val_mae: 1395.9937
121/121 -----
Epoch 40/100      1s 3ms/step - loss: 289187448.0000 - mae: 1255.7841 - val_loss: 345668736.0000 - val_mae: 1390.1458
121/121 -----
Epoch 41/100      1s 4ms/step - loss: 287837776.0000 - mae: 1228.1665 - val_loss: 34564544.0000 - val_mae: 1395.5814
121/121 -----
Epoch 42/100      0s 2ms/step - loss: 183261208.0000 - mae: 1866.9419 - val_loss: 345656648.0000 - val_mae: 1378.2985
121/121 -----
Epoch 43/100      0s 2ms/step - loss: 214183808.0000 - mae: 1283.4282 - val_loss: 34564536.0000 - val_mae: 1392.8742
121/121 -----
Epoch 44/100      0s 2ms/step - loss: 64934080.0000 - mae: 968.3937 - val_loss: 345632992.0000 - val_mae: 1398.2532
121/121 -----
Epoch 45/100      0s 2ms/step - loss: 122387608.0000 - mae: 1858.8838 - val_loss: 34563048.0000 - val_mae: 1411.8552
121/121 -----
Epoch 46/100      1s 3ms/step - loss: 343818448.0000 - mae: 1525.7589 - val_loss: 345653888.0000 - val_mae: 1411.6847
121/121 -----
Epoch 47/100      1s 2ms/step - loss: 388643952.0000 - mae: 1654.6318 - val_loss: 345647048.0000 - val_mae: 1480.2938
121/121 -----
Epoch 48/100      0s 2ms/step - loss: 287679424.0000 - mae: 1438.9213 - val_loss: 345644800.0000 - val_mae: 1485.0888
121/121 -----
Epoch 49/100      1s 2ms/step - loss: 345889128.0000 - mae: 1435.6249 - val_loss: 345639988.0000 - val_mae: 1397.9395
121/121 -----
Epoch 50/100      0s 2ms/step - loss: 175439424.0000 - mae: 1172.4488 - val_loss: 345643296.0000 - val_mae: 1486.7888
121/121 -----
Epoch 51/100      1s 2ms/step - loss: 218278968.0000 - mae: 1198.8879 - val_loss: 345641996.0000 - val_mae: 1489.4672
121/121 -----
Epoch 52/100

```

Figure 5.2.2 Epoch 25-52

```

Epoch 53/100      0s 2ms/step - loss: 426032672.0000 - mae: 1737.6836 - val_loss: 345636168.0000 - val_mae: 1418.2946
121/121 -----
Epoch 54/100      0s 2ms/step - loss: 188823776.0000 - mae: 1184.7988 - val_loss: 345627328.0000 - val_mae: 1482.5857
121/121 -----
Epoch 55/100      0s 2ms/step - loss: 622784576.0000 - mae: 2878.7884 - val_loss: 345627296.0000 - val_mae: 1484.4541
121/121 -----
Epoch 56/100      0s 2ms/step - loss: 388249888.0000 - mae: 1577.3955 - val_loss: 345623744.0000 - val_mae: 1399.4284
121/121 -----
Epoch 57/100      0s 2ms/step - loss: 177921184.0000 - mae: 1288.8583 - val_loss: 345611848.0000 - val_mae: 1387.8338
121/121 -----
Epoch 58/100      0s 2ms/step - loss: 423889856.0000 - mae: 1698.5688 - val_loss: 345631616.0000 - val_mae: 1412.8016
121/121 -----
Epoch 59/100      0s 2ms/step - loss: 18341186.0000 - mae: 1181.1365 - val_loss: 345638496.0000 - val_mae: 1418.5129
121/121 -----
Epoch 60/100      0s 2ms/step - loss: 219878948.0000 - mae: 1285.2229 - val_loss: 345624880.0000 - val_mae: 1411.5849
121/121 -----
Epoch 61/100      1s 2ms/step - loss: 624612096.0000 - mae: 2861.8892 - val_loss: 345624448.0000 - val_mae: 1489.8875
121/121 -----
Epoch 62/100      1s 2ms/step - loss: 214559592.0000 - mae: 1237.6244 - val_loss: 345621216.0000 - val_mae: 1486.5358
121/121 -----
Epoch 63/100      0s 2ms/step - loss: 269389344.0000 - mae: 1371.8337 - val_loss: 345618528.0000 - val_mae: 1489.8368
121/121 -----
Epoch 64/100      1s 2ms/step - loss: 129217344.0000 - mae: 1147.5925 - val_loss: 345616112.0000 - val_mae: 1486.9788
121/121 -----
Epoch 65/100      0s 2ms/step - loss: 373381584.0000 - mae: 1514.4153 - val_loss: 345618272.0000 - val_mae: 1395.1782
121/121 -----
Epoch 66/100      1s 4ms/step - loss: 281427588.0000 - mae: 1232.6782 - val_loss: 345617888.0000 - val_mae: 1424.1715
121/121 -----
Epoch 67/100      1s 5ms/step - loss: 187864884.0000 - mae: 1268.7271 - val_loss: 345618272.0000 - val_mae: 1419.5764
121/121 -----
Epoch 68/100      0s 3ms/step - loss: 414135552.0000 - mae: 1852.9298 - val_loss: 345609792.0000 - val_mae: 1483.7859
121/121 -----
Epoch 69/100      1s 4ms/step - loss: 157855232.0000 - mae: 1138.5743 - val_loss: 345611184.0000 - val_mae: 1428.8887
121/121 -----
Epoch 70/100      1s 7ms/step - loss: 226166888.0000 - mae: 1287.7783 - val_loss: 345608816.0000 - val_mae: 1419.5582
121/121 -----
Epoch 71/100      1s 4ms/step - loss: 265411536.0000 - mae: 1399.1821 - val_loss: 345611744.0000 - val_mae: 1421.9744
121/121 -----
Epoch 72/100      0s 2ms/step - loss: 138292768.0000 - mae: 1808.8863 - val_loss: 345606432.0000 - val_mae: 1413.9888
121/121 -----
Epoch 73/100      0s 2ms/step - loss: 183484172.0000 - mae: 1836.7187 - val_loss: 345609584.0000 - val_mae: 1428.4853
121/121 -----
Epoch 74/100      1s 2ms/step - loss: 219251856.0000 - mae: 1261.6217 - val_loss: 345608288.0000 - val_mae: 1423.1793
121/121 -----
Epoch 75/100      1s 2ms/step - loss: 194549928.0000 - mae: 1212.7169 - val_loss: 345598176.0000 - val_mae: 1423.3885
121/121 -----
Epoch 76/100      0s 3ms/step - loss: 129393848.0000 - mae: 1181.9243 - val_loss: 345605344.0000 - val_mae: 1413.9141
121/121 -----
Epoch 77/100      1s 2ms/step - loss: 379256288.0000 - mae: 1561.1578 - val_loss: 345598616.0000 - val_mae: 1419.2188
121/121 -----
Epoch 78/100      0s 2ms/step - loss: 266671376.0000 - mae: 1457.2893 - val_loss: 345608448.0000 - val_mae: 1427.5989
121/121 -----
Epoch 79/100      0s 2ms/step - loss: 284638512.0000 - mae: 1384.8206 - val_loss: 345594888.0000 - val_mae: 1414.9124
121/121 -----
Epoch 80/100      0s 2ms/step - loss: 247719824.0000 - mae: 1389.8475 - val_loss: 345582876.0000 - val_mae: 1417.7821
121/121 -----
Epoch 81/100

```

Figure 5.2.3 Epoch 53-80

```

Epoch 81/100      0s 2ms/step - loss: 158332352.0000 - mae: 1134.2563 - val_loss: 345595984.0000 - val_mae: 1424.8193
121/121 -----
Epoch 82/100      0s 2ms/step - loss: 171668064.0000 - mae: 1178.8589 - val_loss: 345593680.0000 - val_mae: 1433.8918
121/121 -----
Epoch 83/100      0s 2ms/step - loss: 422795552.0000 - mae: 1783.3914 - val_loss: 345598816.0000 - val_mae: 1427.6493
121/121 -----
Epoch 84/100      1s 2ms/step - loss: 338968256.0000 - mae: 1476.8438 - val_loss: 345589288.0000 - val_mae: 1422.4877
121/121 -----
Epoch 85/100      0s 2ms/step - loss: 196288944.0000 - mae: 1511.8238 - val_loss: 345583528.0000 - val_mae: 1417.7369
121/121 -----
Epoch 86/100      0s 2ms/step - loss: 481296768.0000 - mae: 1833.7244 - val_loss: 345598336.0000 - val_mae: 1427.3851
121/121 -----
Epoch 87/100      0s 2ms/step - loss: 573245444.0000 - mae: 1998.1565 - val_loss: 345588784.0000 - val_mae: 1422.6576
121/121 -----
Epoch 88/100      1s 2ms/step - loss: 158833528.0000 - mae: 1129.5089 - val_loss: 345588576.0000 - val_mae: 1425.1984
121/121 -----
Epoch 89/100      0s 2ms/step - loss: 129183152.0000 - mae: 1854.4454 - val_loss: 345583296.0000 - val_mae: 1429.8829
121/121 -----
Epoch 90/100      0s 2ms/step - loss: 344334240.0000 - mae: 1574.2546 - val_loss: 345578176.0000 - val_mae: 1425.3603
121/121 -----
Epoch 91/100      0s 2ms/step - loss: 386681928.0000 - mae: 1478.7062 - val_loss: 345578368.0000 - val_mae: 1427.9393
121/121 -----
Epoch 92/100      0s 2ms/step - loss: 244737872.0000 - mae: 1459.2966 - val_loss: 345574272.0000 - val_mae: 1422.6595
121/121 -----
Epoch 93/100      0s 2ms/step - loss: 135892976.0000 - mae: 1118.1877 - val_loss: 345574848.0000 - val_mae: 1411.1752
121/121 -----
Epoch 94/100      0s 2ms/step - loss: 125177752.0000 - mae: 1881.4341 - val_loss: 345572160.0000 - val_mae: 1432.8586
121/121 -----
Epoch 95/100      1s 2ms/step - loss: 37754464.0000 - mae: 1711.8796 - val_loss: 345572128.0000 - val_mae: 1438.6564
121/121 -----
Epoch 96/100      0s 2ms/step - loss: 177614944.0000 - mae: 1282.7867 - val_loss: 345568288.0000 - val_mae: 1424.9382
121/121 -----
Epoch 97/100      0s 2ms/step - loss: 381592224.0000 - mae: 1422.8598 - val_loss: 345567328.0000 - val_mae: 1434.8383
121/121 -----
Epoch 98/100      0s 2ms/step - loss: 195788144.0000 - mae: 1238.6389 - val_loss: 345565184.0000 - val_mae: 1432.9296
121/121 -----
Epoch 99/100      0s 3ms/step - loss: 268526848.0000 - mae: 1379.5675 - val_loss: 345565688.0000 - val_mae: 1436.3176
121/121 -----
Epoch 100/100     0s 3ms/step - loss: 174768912.0000 - mae: 1254.2689 - val_loss: 345563616.0000 - val_mae: 1436.3661
Model training completed successfully.

```

Figure 5.2.4 Epoch 81-100

From figure 5.2.1 to 5.2.4 it explains the epoch levels of 1-100 which is used for better accuracy. Iteratively improving a neural network model's performance by modifying its internal parameters is the goal of running the model through epochs 1 through 100. The model can discover patterns by going over the full training dataset once for each epoch. The model's performance is indicated by the values related to each epoch, such as training loss and validation loss. As the model learns, it should be ideal for both training and validation loss to decrease. On the other hand, overfitting is indicated if the validation loss begins to rise or plateau while the training loss is declining. This indicates that rather than effectively generalising to new data, the model is just memorising the training set. Overfitting

results in poorer performance on fresh, untested data. When the validation loss is at its maximum and may be different from the training loss, the performance is at its poorest. Finding this moment is essential for halting the training process to avoid overfitting and guarantee that the model generalises well. We can identify the sweet spot where the model performs well on both training and validation data by tracking the loss values over epochs.

```
Test Loss (Mean Squared Error): 4932798976.0000
Test Mean Absolute Error (MAE): 3351.9917
```

Figure 5.2.5

In figure 5.2.5 it is shown that the mean squared error (MSE) is 4,932,798,976 and the test mean absolute error (MAE) is 3,351.9917 which indicated that the MSE is actually very high and not suitable as its performance and predictions have a large variance. The MSE value substantially deviates from the actual target values.

```
38/38 ————— 0s 2ms/step
R2 Score after First Epoch: 0.0029
```

Figure 5.2.6 First epoch

The model's first epoch results are shown in figure 5.2.6, in just a few seconds, the model ran 38 batches of data, spending 2 milliseconds per batch on average. The model performed extremely badly, according to its R-squared score of 0.0029, which tells us it only managed to explain a tiny fraction of variance in the target variable.

```
R2 Score at Mid Epoch (Epoch 51): 0.0029
```

Figure 5.2.7 Mid epoch

Approximately halfway through training in figure 5.2.7, by counting all the epochs and dividing it by two we can get the mid epoch. The model shows in the output that its R-squared rating was very low even by epoch 51, meaning it didn't learn much better during the first half of training. The model did not grow and improve its ability to predict during the first half of its training process.

```
Best Epoch R2 Score (Epoch 100): 0.0029
```

Figure 5.2.8 Best epoch

At the last epoch (100) in figure 5.2.8, the model's highest R-squared score was still at 0.0029. This indicates that the predicted accuracy did not increase over the course of the training session. This kind of stalling suggests that there may be deeper problems with the model's design, feature selection, data quality, or hyperparameter settings.

```
Overall R2 Score (Accuracy): 0.0029
```

Figure 5.2.9 the overall R2

R-squared score is a statistical measure used in figure 5.2.9 that represents the proportion of variance in the target variable which indicates whether the model fits the data. Since the overall score is only 0.0029, relatively close to 0, this makes the NN model not very suitable.

5.3 K-Means Clustering Experiments

This experiment investigates how varying the number of clusters influences model performance from a baseline model where $k=3$ to the best performing where $k=10$ and finally to intermediate number of clusters, $k=6$. The results provide insight on the best clustering configuration and will also focus the importance of hyperparameter tuning in finding meaningful and interpretable groupings.

1. First Model (k=3)

First R-squared accuracy: 0.1550

Figure 5.3.1 First R-squared accuracy

Figure 5.3.1 shows the R-squared accuracy for this first model was 0.1550. This low accuracy shows that 3 clusters are not enough to show the structure of the dataset.

2. Best Model (k=10)

Tuned R-squared accuracy (best k=10): 0.5551

Figure 5.3.2 Tuned R-squared accuracy

Figure 5.3.2 displays the R-squared accuracy increased significantly to 0.5551 when the number of clusters increased to 10. Therefore, this indicates that 10 clusters are better fit for the data. Hence, explaining more variance and capturing more fine-grained patterns.

3. Intermediate Model (k=6)

Intermediate R-squared accuracy (k=6): 0.5157

Figure 5.3.3 Intermediate R-squared accuracy

Figure 5.3.3 shows the R-squared accuracy was 0.5157. While already an improvement over the first model, being slightly lower than the best model highlights that 6 clusters are still suboptimal for this dataset.

5.4 Gradient Boosting Experiment

```
Accuracy with learning_rate=0.1, n_estimators=200, max_depth=9, min_samples_split=10, subsample=1.0: 0.78
Accuracy with learning_rate=0.1, n_estimators=200, max_depth=7, min_samples_split=2, subsample=0.8: 0.80
Accuracy with learning_rate=0.1, n_estimators=200, max_depth=7, min_samples_split=2, subsample=1.0: 0.80
Accuracy with learning_rate=0.1, n_estimators=200, max_depth=7, min_samples_split=10, subsample=0.8: 0.80
Accuracy with learning_rate=0.2, n_estimators=50, max_depth=3, min_samples_split=2, subsample=0.8: 0.71
Accuracy with learning_rate=0.2, n_estimators=50, max_depth=3, min_samples_split=2, subsample=1.0: 0.72
Accuracy with learning_rate=0.2, n_estimators=50, max_depth=3, min_samples_split=10, subsample=0.8: 0.72
Accuracy with learning_rate=0.2, n_estimators=50, max_depth=3, min_samples_split=10, subsample=1.0: 0.72
Accuracy with learning_rate=0.2, n_estimators=50, max_depth=5, min_samples_split=2, subsample=0.8: 0.76
Accuracy with learning_rate=0.2, n_estimators=50, max_depth=5, min_samples_split=2, subsample=1.0: 0.76
Accuracy with learning_rate=0.2, n_estimators=50, max_depth=5, min_samples_split=10, subsample=0.8: 0.76
Accuracy with learning_rate=0.2, n_estimators=50, max_depth=5, min_samples_split=10, subsample=1.0: 0.76
Accuracy with learning_rate=0.2, n_estimators=50, max_depth=7, min_samples_split=2, subsample=0.8: 0.78
Accuracy with learning_rate=0.2, n_estimators=50, max_depth=7, min_samples_split=2, subsample=1.0: 0.78
Accuracy with learning_rate=0.2, n_estimators=50, max_depth=7, min_samples_split=10, subsample=0.8: 0.78
Accuracy with learning_rate=0.2, n_estimators=50, max_depth=7, min_samples_split=10, subsample=1.0: 0.78
Accuracy with learning_rate=0.2, n_estimators=100, max_depth=3, min_samples_split=2, subsample=0.8: 0.75
Accuracy with learning_rate=0.2, n_estimators=100, max_depth=3, min_samples_split=2, subsample=1.0: 0.75
Accuracy with learning_rate=0.2, n_estimators=100, max_depth=3, min_samples_split=10, subsample=0.8: 0.75
Accuracy with learning_rate=0.2, n_estimators=100, max_depth=3, min_samples_split=10, subsample=1.0: 0.75
Accuracy with learning_rate=0.2, n_estimators=100, max_depth=5, min_samples_split=2, subsample=0.8: 0.78
Accuracy with learning_rate=0.2, n_estimators=100, max_depth=5, min_samples_split=2, subsample=1.0: 0.78
Accuracy with learning_rate=0.2, n_estimators=100, max_depth=5, min_samples_split=10, subsample=0.8: 0.79
Accuracy with learning_rate=0.2, n_estimators=100, max_depth=5, min_samples_split=10, subsample=1.0: 0.79
Accuracy with learning_rate=0.2, n_estimators=100, max_depth=7, min_samples_split=2, subsample=0.8: 0.80
Accuracy with learning_rate=0.2, n_estimators=100, max_depth=7, min_samples_split=2, subsample=1.0: 0.80
Accuracy with learning_rate=0.2, n_estimators=100, max_depth=7, min_samples_split=10, subsample=0.8: 0.80
Accuracy with learning_rate=0.2, n_estimators=100, max_depth=7, min_samples_split=10, subsample=1.0: 0.80
Accuracy with learning_rate=0.2, n_estimators=200, max_depth=3, min_samples_split=2, subsample=0.8: 0.77
Accuracy with learning_rate=0.2, n_estimators=200, max_depth=3, min_samples_split=2, subsample=1.0: 0.77
Accuracy with learning_rate=0.2, n_estimators=200, max_depth=3, min_samples_split=10, subsample=0.8: 0.77
Accuracy with learning_rate=0.2, n_estimators=200, max_depth=3, min_samples_split=10, subsample=1.0: 0.77
Accuracy with learning_rate=0.2, n_estimators=200, max_depth=5, min_samples_split=2, subsample=0.8: 0.80
Accuracy with learning_rate=0.2, n_estimators=200, max_depth=5, min_samples_split=2, subsample=1.0: 0.80
Accuracy with learning_rate=0.2, n_estimators=200, max_depth=5, min_samples_split=10, subsample=0.8: 0.81
Accuracy with learning_rate=0.2, n_estimators=200, max_depth=5, min_samples_split=10, subsample=1.0: 0.81
Accuracy with learning_rate=0.2, n_estimators=200, max_depth=7, min_samples_split=2, subsample=0.8: 0.81
Accuracy with learning_rate=0.2, n_estimators=200, max_depth=7, min_samples_split=2, subsample=1.0: 0.81
Accuracy with learning_rate=0.2, n_estimators=200, max_depth=7, min_samples_split=10, subsample=0.8: 0.81
Accuracy with learning_rate=0.2, n_estimators=200, max_depth=7, min_samples_split=10, subsample=1.0: 0.81
```

Figure 5.4.1 R-squared accuracy results

```
Best Parameters: (0.2, 200, 7, 2, 0.8)
Best Accuracy: 0.8091077602854925
```

Figure 5.4.2 Best Accuracy from the results above

Figure 5.4.1 illustrates the impact of various hyperparameter settings on the performance of the Gradient Boosting algorithm. As for the learning rate, which controls how much each tree contributes, values in the middle to high range, like 0.1 and 0.2, produce better accuracy than low values like 0.01,

so fast learning is best for this dataset. The number of estimators controls the number of sequential trees. Models with more trees like 200 will do better than models with fewer trees like 50 because they can learn more complex structures. The maximum depth of the trees is also important. Trees with a deeper structure like max_depth=7 are more accurate than trees with a shallow structure like max_depth=3 or 5. This is likely because a larger model can show how values in the dataset are related in more complex ways. Moreover, a lower value for the minimum samples to split a node will yield slightly better performance than a high value like min_samples_split=10 as it allows for a greater number of splits and a greater granularity between them. Finally, the subsample parameter controls the number of samples for each tree. It shows that using the whole dataset (subsample = 1.0) will give better results than using it in a proportional way like subsample=0.8.

Figure 5.4.2 shows the best configuration for this dataset is a 0.1 or 0.2 value for the learning rate, 200 estimators, a 7 value for a max depth, a 2 value for a min_samples_split, and a 1.0 value for a subsample. All these observations reveal the significance of hyperparameter values in getting the best model performance and provide a strong direction for training a more robust and accurate gradient-boosting model.

6.0 Analysis & Discussion

6.1 Evaluation of the Random Forest Model

The final Random Forest model was evaluated using the test set, achieving an accuracy of 86.69%. Further understanding of the model performance emerged through its classification report combination with confusion matrix. The RF model demonstrated strong performance for detecting respectively frequent rental categories (0 and 1) with over 80% precision and recall scores. The model demonstrated exceptional reliability with category 0 (rentals below RM1500) measuring precision at 0.90 and recall at 0.93. Similarly, category 1 (rentals between RM1500 and RM3000) achieved precision and recall values of 0.82 each.

However, performance dropped significantly for less frequent categories (2, 3, and 4). For example, category 2 (rentals between RM3000 and RM4500) had a precision of 0.75 and a recall of only 0.43, resulting in an F1-score of 0.55. The two least prevalent categories, along with their high precision rates, experienced low recall because they contained a limited number of training samples which intensified the difficulty from class imbalanced data.

Table 6.1.1: Classification Report

Class	Precision	Recall	F1-Score	Support
0	0.90	0.93	0.92	2349
1	0.82	0.82	0.82	1438
2	0.75	0.43	0.55	158
3	1.00	0.29	0.45	31
4	0.77	0.45	0.57	22

Table 6.1.1 shows the classification results show high performance in categories 0 and 1 yet the model faces difficulties predicting rare class categories.

The top 20 most important features contributing to the model's predictions were analysed. As shown in Figure 6.1.1, apartment size, completion year, and the number of rooms were the most influential factors in determining rental prices. The analysis identified building size together with construction date as the strongest predictive factors due to the widely held belief that newer and larger apartments command higher rental prices. The model's predictions showed significant influence from additional factors including bathrooms along with parking spaces along with the main contributing elements.

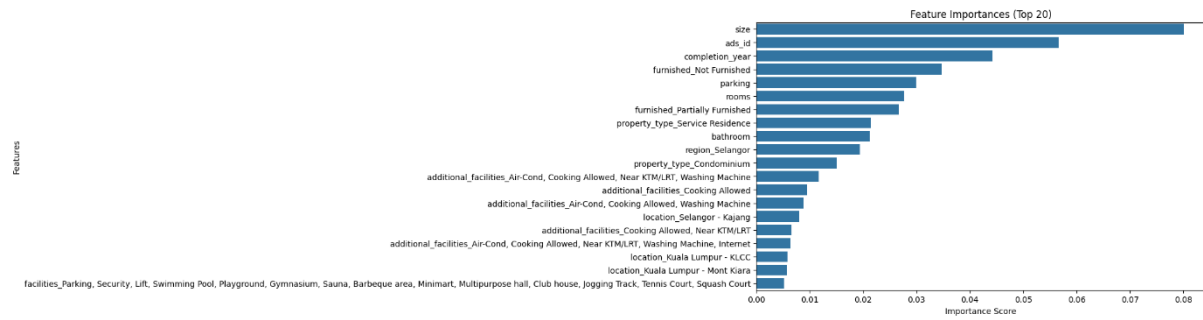


Figure 6.1.1: Feature Importances (Top 20)

Figure 6.1.1 highlights the importance of various features. Rental prices rely heavily on structural elements such as size and rooms and bathroom numbers, together with location-based factors.

The experimental results along with feature analysis generate multiple significant theoretical and practical outcomes. The Random Forest classifier demonstrates successful rental price category prediction with 86.69% accuracy among frequent classes 0 and 1. Making improvements to the prediction models for the less occurring classes 2, 3, and 4 remains a vital priority. Cost-sensitive learning together with oversampling and synthetic data generation techniques through SMOTE can improve the model's ability to operate on minority cases.

The feature importance analysis reveals practical guidance for real estate sector stakeholders. Physical characteristics of apartment size combined with age and room count strongly impact the rental prices variables according to this analysis (Hu et al., 2023). Market developments can use this research to shape their pricing strategies through optimized focus on important rental features.

Lastly, the low minority class recall indicates a need for both more refined data or alternative modelling approaches to address the issue of class imbalance. Using ensemble methods in conjunction with hybrid algorithms shows potential to provide better modelling performance and equal treatment across the whole range of rental categories.

In short, the Random Forest algorithm maintained a high level of performance when identifying rental price categories determined by apartment features. The model proved effective at identifying frequently occurring categories but additional improvements through class imbalance correction and advanced technique implementation would boost its practical applicability. The enhanced model shows strong potential for practical applications within the real estate industry.

6.2 Evaluation of the Neural Network Model

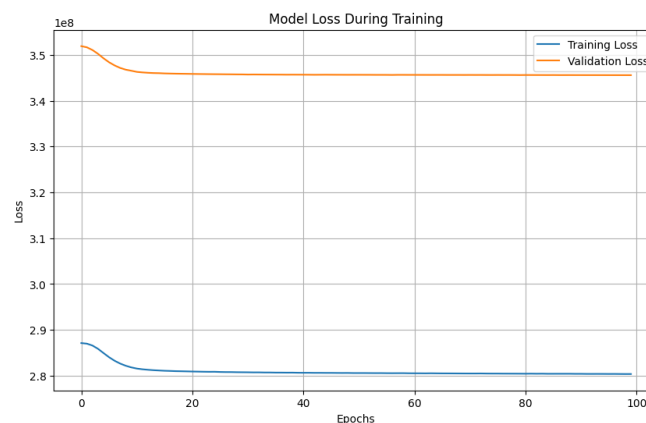


Figure 6.2.1 Model loss during training

Figure 6.2.1 is a visual of the training and validation loss curves across 100 epochs. On the x-axis is the epochs numbers and on the y-axis is the number of losses. The training loss line in blue line represents how well the model fits the data it was trained on, while the validation loss line in orange measures how well the model generalizes (avoiding overfitting). Both the curves start from a high loss value of about 3.5, which indicates the model's initial predictions are far from being accurate. For the training loss process, both losses decrease steadily and reach approximately 2.8 by the 100th epoch. The slowing rate of loss reduction in the later epochs (beyond epoch 60) indicates that the model has reached its performance limit.

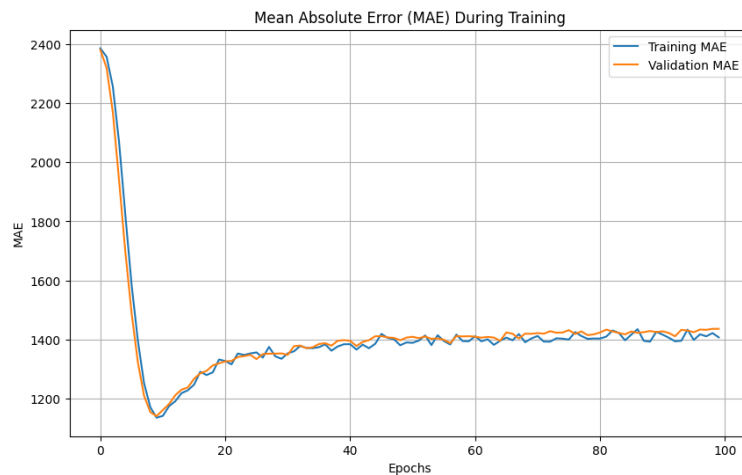


Figure 6.2.2 Mean Absolute Error (MAE) during training

This statistical plot figure 6.2.2 shows through multiple lines how neural network training affected model learning together with performance assessment data points across 100 epochs. The y-axis shows MAE to measure the average absolute value differences between predicted and actual results while epochs along the x-axis represent full training data passes through the model. The starting point uses uninitiated parameters to produce tremendously inaccurate predictions which appear as high initial training MAE (blue line) and validation MAE (orange line) at 2400. The substantial learning period of the model starts during the initial 20 epochs through which the model learns to adjust its weights and biases. The MAE reaches stability at 1400 marking the model's performance plateau which occurs when training continues for additional epochs beyond which the model demonstrates no further significant performance growth. The model achieved maximum performance within its current limits because of architectural choices as well as data types and parameter values. A plateau appears in the graph after around 20 epochs resulting in 1400 being the stabilized MAE measurement for both the training and validation datasets while showing minimal random deviations. The currently responsible model fails to gain meaningful knowledge from the data which demonstrates it has maximized its existing constraints. Proficiency in overcoming plateaus requires modifications both to architecture and parameter values while adding enhanced training datasets. The model shows strong generalization qualities through its continuous performance similarity on both unseen validation data sets and training data sets which maintain a minimal MAE gap during the entire execution process. The near parallel curves in this model demonstrate excellent equilibrium between training and testing data without evidence of major overfitting. The optimization potential of the model seems limited by architectural constraints and dataset and hyperparameter factors even though additional sophisticated techniques could improve both accuracy and overall performance.

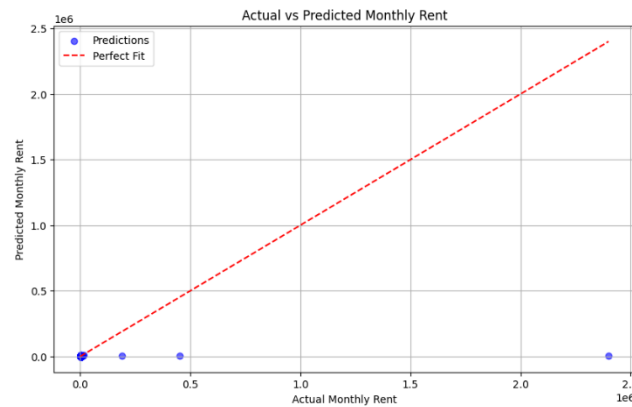


Figure 6.2.3 Actual Vs. Predicted monthly rent

Figure 6.2.3 displays how the neural network model predicts apartment rentals when compared to real rental data. Figures on x-axis demonstrate real rent distributions to compare them with model predictions shown on y-axis. The red dashed line functions as the "perfect fit" line which indicates that ideal model performance would result in every blue dot converging perfectly onto this line. Most blue points in this graph deviate significantly from the red prediction line particularly when measuring higher rent values. The modelling system predicts a value near zero for estimates exceeding 2.5 million in actual rental prices while simultaneously demonstrating prediction errors for large values. Point distribution clusters toward the bottom left section revealing the model delivers low values across all rent ranges. The model fails to detect meaningful relationships between variables during estimation processes specifically for rent amounts above RM2.5 million.

6.3 Evaluation of K-Means Clustering Model

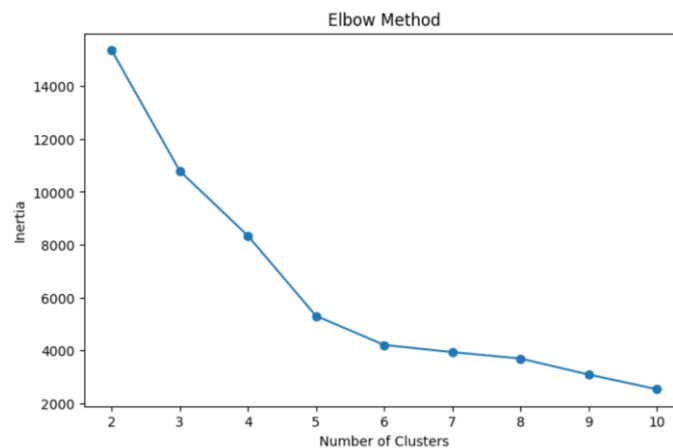


Figure 6.3.1 Elbow Method

Figure 6.3.1 shows the relationship between number of clusters (k) and inertia to measure the cluster compactness. It shows an inertia drop in a sharp decline from $k=2$ to $k=4$ and then gradually curves up. At around $k=4$ or $k=5$, the slope of the reduction rate drops much more significantly, thereby giving an indication of being the elbow point. This points out that $k=4$ or $k=5$ might be the optimal numbers of clusters in the dataset. In our experiment, the best model used $k=10$, shows that the data is too complex for additional clusters to capture most of the useful patterns. Hence, this supports the importance of using the Elbow Method along with other evaluation metrics such as R-squared accuracy for selecting the optimal value of k .

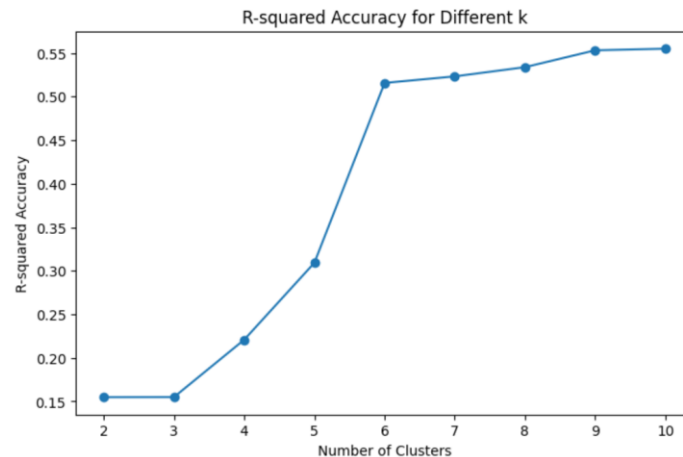


Figure 6.3.2 R-squared accuracy for Different k

Figure 6.3.2 describes the performance of K-Means Clustering model differs with the number of clusters (k). It is observed from the plot that the R-squared accuracy increases to increase with k, reaching its maximum value at 0.5551 for k=10. This suggests that 10 clusters are relatively the best to describe most of the variance in the data. However, this result is inconsistent with the suggestion by Elbow Method based on inertia, where it was optimal either at k=4 or k=5. Various metrics for evaluation must be considered in choosing the optimal value of k. Though the value of k=10 maximizes R-squared accuracy, it increases model complexity and results in overfitting. Therefore, k selection should be a trade-off between performance and interpretability.

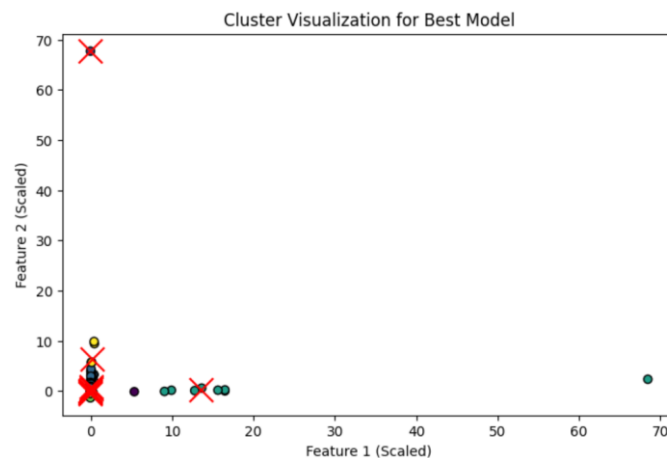


Figure 6.3.3 Cluster Visualization for Best Model

Figure 6.3.3 shows the best model k=10 splits the data into 10 distinct clusters represented using different colours; the red 'X' markers will outline the centroids for every cluster, the central point around which a data point is collected. Based on the figure, it is observable that most of the clusters are quite separated. This agrees with the high R-squared accuracy, 0.5551, realized by the model and supports the fact that the clusters have captured important data patterns well. However, some of these clusters are apparently much closer to one another, showing that the corresponding data points tend to share similar characteristics. In summary, this figure shows the effectiveness of K-Means algorithm was in clustering this dataset into distinguishable and meaningful clusters.

6.4 Evaluation of Gradient Boosting Algorithm

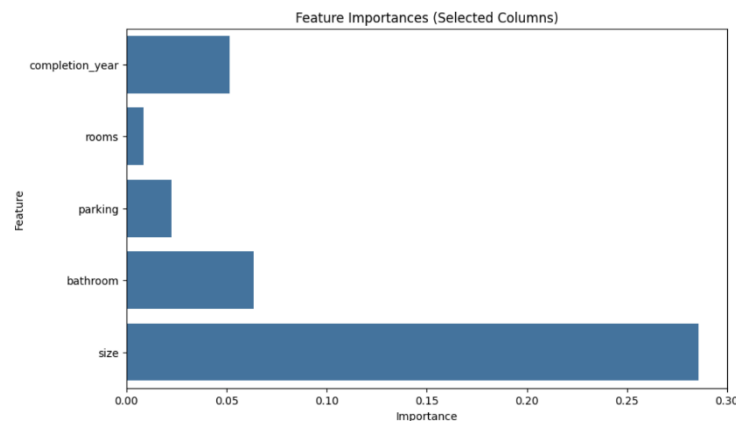


Figure 6.4.1 Feature Importances on selected columns

Figure 6.4.1 shows the comparative contribution of each feature to model prediction. The most important feature is size with a value of 0.285732 and it indicates property size is most significant in the prediction of the target variable value or predicted value. Other than that, feature importance stands at 0.063706 for bathrooms, and it indicates property bedrooms contribute a lot to model prediction. Completion year, parking, and room features have smaller but significant values, and they indicate a lesser but significant contribution towards model prediction. It is beneficial knowing which feature factors drive model prediction and can inform feature selection or engineering in future development.

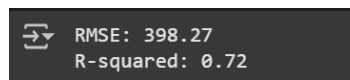


Figure 6.4.2 Model Performance Metrics

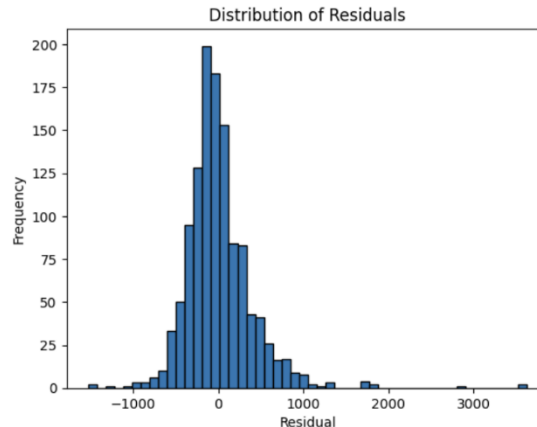


Figure 6.4.3 Distribution of Residuals

Figure 6.4.1 and Figure 6.4.2 shows a general picture of model efficiency constructed with gradient boosting. The histogram affirms residuals focused in and about zero with symmetric distribution, affirming model prediction to be near free of bias and errors happening in even distribution about actual ones. Besides, an existence of outliers in direction of high values, is a sign model is having problem with such specific points generating larger prediction errors. It can be a sign a model is not identifying specific trends or nuance in a model alone. Model performance statistics including an RMSE value of 398.27 and an R-squared value of 0.72 give a numerical analysis of its accuracy. RMSE is a sign of mean value of prediction errors, a lower value being ideal, and an R-squared value of 0.72 a sign of 72% variance in target variable can be produced through model explanations. It describes a pretty good fit but documents room for improvement. Mitigation of such outliers through additional analysis of data, feature engineering, and model fine-tuning can reduce RMSE and ideally maximize an R-squared value, and, in return, an overall model prediction accuracy can be maximized. That

aggregated analysis describes both value in both a visualization and numerical analysis in model performance comprehension and model performance improvement.

6.5 Discussion on the Best Model

Among the four considered models, Neural Network, K-Means Clustering, Random Forest, and Gradient Boosting. The Gradient Boosting model is the best model for this Kuala Lumpur and Selangor rental price dataset. The premise of its conclusion is its high-performance metric, interpretability, and performance in handling complexity in the Kuala Lumpur and Selangor rental price dataset. This model's R-squared value of 0.72 explains 72% of the variation in the target variable (monthly rent price). This is a much higher value than the neural network's 0.0029 and the K-means clustering model's best 0.5551 fit for R-squared accuracy. In contrast, the model's Root Mean Squared Error (RMSE of 398.27) is relatively low, a sign that its prediction values closely mimic real rental price values. With such, Gradient Boosting is a reliable choice for real use, for it is consistently reliable over the dataset.

The Random Forest model, at an accuracy rating of 86.69%, performed poorly when it comes to class imbalance, namely predicting infrequent categories of rentals. On the other hand, Gradient Boosting performed balanced over the dataset with an RMSE of 398.27, somewhat low for such a variance in a rental's dataset. The Random Forest model performs well in predicting most frequent categories of rentals but not for infrequent rentals it achieves low recall for high classes of rentals like 0.43 in Class 2 and 0.29 in Class 3 and low generalizability over the dataset. In contrast, when in a regression form, Gradient Boosting forecasts explicitly continuous rentals values for price with an R-squared value of 0.72 and an RMSE of 398.27, offering actionable and correct information for use by stakeholders. In addition, Gradient Boosting is a model with a deeper feature importance with property size (0.285732) and number of bathrooms (0.063706) being its most significant factors, and for that reason a more interpretative and working model for real-life use cases such as price determination for rentals or cheap housing determination.

One of the advantages of the gradient boosting model is its interpretability and its ability to produce insightful feedback in terms of feature importance. Analysis revealed that property size (0.285732) is most significant in driving rentals, with a closely following number of bedrooms (0.063706). Other factors, such as year of completion, number of parking spaces, and number of rooms, did contribute to driving model prediction, but not as effectively. Such a level of interpretability is beneficial for stakeholders, as it helps them visualize factors driving rentals and make effective decisions. For example, landlords can target larger properties and a high number of bedrooms to maximize rental yield, and tenants can target such an area when searching for affordable housing. In addition, a model residual analysis confirmed that its prediction in general is not biased, with residuals clumped closely together near zero. However, having a few outliers informs us that the model may have an issue with individual observations, and avenues for improvement through feature engineering or hyperparameter tuning can be considered.

In comparison with the other models, gradient boosting works best with sophisticated relationships and non-linear interactions between the features. The Random Forest model performed best especially for less frequently seen categories of rentals were in accuracy in classification but not in class imbalance. The neural network model could not extract meaningful patterns in information, for its high reading of mean squared error (MSE) and low reading of R-squared, as an indicator, inform us about. K-Means the clustering model is a model efficient in clustering similar types of property failed in predictive accuracy in predicting the price of rentals in an apt manner. In contrast, sequential learning in the case of gradient boosting, with one tree striving to correct the errors of its predecessor helps extract sophisticated patterns in information and a model suitable for use with such information.

7.0 Conclusion

The "Rental Pricing Dataset Malaysia" enabled successful analysis of rentals in Kuala Lumpur and Selangor zones in Malaysia. Researchers aimed to discover what factors determine rent costs while identifying appropriate forecast models for different types of rentals.

Our analysis produced crucial data on rental price mechanics which showed property size together with completion year and room count function as main influencing elements. The Random Forest model demonstrated 86.69% accuracy yet provided successful predictions mainly for the first two rental types (Model 0 and Model 1). The model began losing predictive power for rental categories starting from number 2 and extending through to number 4 which demonstrates its reduced effectiveness across multiple price levels.

The Gradient Boosting model produced the best performance when modelling rental price variation because it achieved an R-squared value of 0.72. This model demonstrates strong predictive capabilities throughout all rental categories indicating its potential as a reliable instrument to understand and predict market prices effectively.

Our findings indicate that Random Forest serves as a useful starting point yet Gradient Boosting stands as the optimal selection for this analysis because of its general-purpose accuracy and capability to forecast a broad spectrum of rental price ranges. The research project has found success by expanding our knowledge of Kuala Lumpur/Selangor rentals while generating guidance that benefits real estate stakeholders and local agencies to create a balanced housing market. Further studies should build upon existing model performance and analyze additional rental price influencing factors to support current urban affordable housing strategies.

8.0 Reference

Hu, C., Huang, R., & Li, H. (2023). Prediction and Analysis of Rental Price using Random Forest Machine Learning Technique Take Shanghai and Wuhan for example (pp. 587–593). https://doi.org/10.2991/978-94-6463-042-8_84

Kokasih, M. F., & Suryaputra Paramita, A. (2020). Property Rental Price Prediction Using the Extreme Gradient Boosting Algorithm. *International Journal of Informatics and Information System*, 3(2), 54–59.

Neloy, A. A., Sadman Haque, H. M., & Ul Islam, M. M. (2019). Ensemble learning based rental apartment price prediction model by categorical features factoring. *ACM International Conference Proceeding Series, Part F148150*, 350–356. <https://doi.org/10.1145/3318299.3318377>

Ogundunmade, T. P., Abidoye, M., & Olunfunbi, O. M. (2023). Modelling Residential Housing Rent Price Using Machine Learning Models. *Modern Economy and Management*, 2, 14. <https://doi.org/10.53964/mem.2023014>

Oshodi, O. S., Thwala, W. D., Odubiyi, T. B., Abidoye, R. B., & Aigbavboa, C. O. (2019). Using neural network model to estimate the rental price of residential properties. *Journal of Financial Management of Property and Construction*, 24(2), 217–230. <https://doi.org/10.1108/JFMPC-06-2019-0047>